



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DCCO / Carrera de Software

Sistema de Monitoreo de Integridad de Archivos (FIM)

Prueba de Concepto (PoC FIM) aplicada a Ciberdefensa

Área: Tendencias de Seguridad de la Información

Caso de estudio: Ciberdefensa

Autores: Miranda Alison, Morán David, Vivanco Gabriel

Tutor: Ing. Walter Fuertes

Asignatura: Ingeniería de Seguridad de Software

Fecha: 21/07/2026

Índice

1. Link del Repositorio	2
2. Video Demostración	2
3. Introducción	2
4. Objetivos	2
4.1. Objetivo general	2
4.2. Objetivos específicos	2
5. Alcance del proyecto	3
6. Marco técnico	3
6.1. Tecnologías usadas	3
6.2. Lógica central del proyecto	3
6.3. Arquitectura del sistema	4
6.4. Base de datos	4
7. Implementación	4
7.1. Estructura de archivos	4
7.2. Desarrollo de la solución	4
7.3. Algoritmo de verificación	5
7.4. Flujo de funcionamiento	5
7.5. Sincronización remota	6
8. Pruebas y validación	6
8.1. Requisitos del entorno	6
8.2. Preparación de laboratorio	6
8.3. Pasos de prueba	7
8.4. Verificación de resultados	10
9. Cómo correr el proyecto	10
9.1. Interfaz web	10
9.2. Modo CLI	10
9.3. Ejecución directa en caso de múltiples versiones de Python	10
9.4. Gestión de errores y robustez	11
10. Cómo probar CRUD desde Kali	11
11. Resultados esperados	11
12. Conclusiones	12
13. Recomendaciones futuras	12
14. Anexos: evidencia adicional	12
14.1. Topología del laboratorio	13
14.2. Contenido de la base de datos	13
14.3. Ejecución en modo CLI / headless	13
15. Bibliografía y referencias	13

1. Link del Repositorio

<https://github.com/mirandaalison/Sistema-de-monitoreo-de-integridad.git>

2. Video Demostración

<https://drive.google.com/drive/folders/1Wk73WH7KQTwhbgSPg12KoBVJ352wUft7?usp=sharing>

3. Introducción

El presente informe documenta el desarrollo de una prueba de concepto (PoC) de un sistema de monitoreo de integridad de archivos (FIM) orientado a ciberdefensa. El objetivo es detectar cambios en un entorno de laboratorio, donde Ubuntu Desktop actúa como plataforma defensora y Metasploitable2 es el objetivo monitoreado.

La idea central del proyecto es demostrar una solución defensiva que observe un objetivo comprometido de forma remota, detecte alteraciones en archivos críticos y registre incidentes sin depender de software externo. El sistema separa los roles de forma clara: Ubuntu Desktop realiza el monitoreo, Metasploitable2 es el sistema monitoreado y Kali Linux representa el atacante que modifica los archivos.

Este proyecto aborda elementos clave de ciberdefensa: detección de alteraciones, sincronización remota segura, gestión de línea base y visualización de alertas. Se construyó utilizando únicamente librerías estándar de Python para mantener el enfoque en la lógica, la robustez y la reproducibilidad del resultado técnico.

La prueba de concepto está pensada para demostrar cómo un sistema defensor puede detectar indicadores de compromiso en un objetivo remoto sin instalar agentes en el host monitoreado. Se prioriza la separación de roles y la trazabilidad de cada cambio detectado.

4. Objetivos

4.1. Objetivo general

Desarrollar una PoC de ciberdefensa que detecte, registre y presente cambios en archivos críticos de un sistema remoto vulnerado.

4.2. Objetivos específicos

- Implementar un sensor de integridad basado en hash SHA-256.
- Sincronizar archivos de Metasploitable2 hacia Ubuntu para su análisis.
- Generar alertas de creación, modificación y eliminación.
- Proveer una interfaz local accesible por navegador.
- Documentar el proceso, pruebas y resultados.

5. Alcance del proyecto

El proyecto cubre:

- Monitoreo remoto desde Ubuntu Desktop hacia Metasploitable2.
- Registro de alertas en base de datos SQLite.
- Interfaz web local en <http://127.0.0.1:8000>.
- Operación sin dependencias externas: solo Python estándar.

No incluye:

- Detección de atacante por identidad.
- Autenticación inherente a la aplicación.
- Mecanismos de mitigación automática.

6. Marco técnico

6.1. Tecnologías usadas

- Python 3
- SQLite
- `scp` y `ssh` para sincronización remota
- Librerías estándar de Python: `os`, `sqlite3`, `hashlib`, `time`, `datetime`, `threading`, `subprocess`, `http.server`, `tkinter` (opcional)

6.2. Lógica central del proyecto

El proyecto se basa en las siguientes ideas principales:

- El sistema defensor no monitorea directamente el sistema atacado; en su lugar, sincroniza una copia de la carpeta crítica desde el objetivo remoto para evitar el monitoreo local de Windows.
- La integridad se comprueba con hashes SHA-256 que se comparan con una línea base almacenada en SQLite.
- El proceso de detección distingue tres tipos de eventos: creación de archivos, modificación de archivos y eliminación de archivos.
- La detección se realiza de forma periódica o bajo demanda, y los resultados se muestran en una interfaz local accesible por navegador.
- El diseño enfatiza la simplicidad, la trazabilidad y la independencia de dependencias externas para un entorno académico.

6.3. Arquitectura del sistema

- **Ubuntu Desktop** es el host defensor.
- **Metasploitable2** es el host monitoreado.
- **Kali Linux** se usa como actor atacante para modificar el objetivo.
- La carpeta remota `/home/msfadmin/carpeta_critica` se sincroniza hacia `~/fim_poc/archivos_criticos`.

6.4. Base de datos

La aplicación usa SQLite con dos tablas principales:

- **inventario**: almacena `ruta_archivo`, `hash_seguro` y `ultima_verificacion`.
- **alertas**: registra cambios con `ruta_archivo`, `fecha_hora`, `usuario`, `hash_anterior`, `hash_nuevo` y `tiempo_deteccion_ms`.

7. Implementación

7.1. Estructura de archivos

- `fim_poc.py`: aplicación principal con lógica de línea base, escaneo, sincronización remota e interfaz web.
- `read_alertas.py`: script auxiliar para consultar alertas desde la base de datos.
- `README.md`: documentación de uso del proyecto.
- `INFORME_PROYECTO.md`: este informe.

7.2. Desarrollo de la solución

El código central se organiza en bloques funcionales claramente diferenciados. El archivo `fim_poc.py` contiene:

- Inicialización del entorno y base de datos con `init_db()`.
- Creación y mantenimiento de la carpeta de vigilancia local con `ensure_watch_dir()`.
- Lectura y cálculo de hash SHA-256 en `sha256_of_file()`. Esta función procesa archivos por bloques para manejar ficheros grandes.
- Sincronización remota con `sincronizar_remoto()`, que copia los datos remotos en un directorio temporal y luego actualiza el directorio local de observación.
- Gestión de la línea base con `establecer_linea_base()`, que guarda hashes iniciales en la tabla `inventario`.
- Verificación de integridad con `verificar_integridad()`, que compara el estado actual con la línea base y registra alertas cuando los hashes cambian.
- Registro de alertas en la tabla `alertas` mediante `insertar_alerta()`.
- Interfaz web local implementada con `BaseHTTPRequestHandler` y `ThreadingHTTPServer`.

- Un modo de ejecución headless y un componente de monitoreo continuo para escaneos periódicos.

7.3. Algoritmo de verificación

La función `verificar_integridad()` ejecuta el algoritmo crítico de comparación:

1. Se carga la línea base desde la tabla `inventario` y se normalizan las rutas de archivo para evitar discrepancias de formato.
2. Si no existe línea base previa, se genera automáticamente con `establecer_linea_base()` y no se producen alertas.
3. Se sincroniza el contenido remoto desde Metasploitable2 y solo si esta operación es exitosa se procede con el escaneo.
4. Se recorre la copia local actual de `archivos_criticos` y se calcula el hash actual de cada archivo.
5. Para cada archivo:
 - Si el archivo existe en la línea base y el hash es diferente, se inserta una alerta de modificación y se actualiza el inventario.
 - Si el archivo existe en la línea base y el hash es igual, se actualiza la marca de última verificación sin generar alerta.
 - Si el archivo no existía en la línea base, se inserta una alerta de creación y se agrega al inventario.
6. Se detectan eliminaciones calculando la diferencia entre los conjuntos de rutas base y actuales. Cada archivo eliminado genera una alerta y se borra del inventario.

7.4. Flujo de funcionamiento

La implementación sigue un flujo claramente definido para asegurar que el monitoreo sea confiable y reproducible:

1. **Inicialización:** el sistema crea el directorio de trabajo local en `~/fim_poc` y prepara la base de datos SQLite.
2. **Establecer Línea Base:** se escanean los archivos presentes en el directorio local de vigilancia y se calculan hashes SHA-256. Estos hashes se guardan en la tabla `inventario` como referencia para futuras comparaciones.
3. **Sincronizar Remoto:** antes de cada verificación, el sistema copia el contenido de la carpeta remota en Metasploitable2 hacia un directorio temporal local. El directorio local final se actualiza para reflejar el contenido remoto, incluyendo la eliminación de archivos que ya no existen en el objetivo.
4. **Escanear Ahora:** el sistema recorre la copia local sincronizada, calcula el hash de cada archivo y compara contra la línea base.
5. **Generación de alertas:** se registra un evento en la tabla `alertas` cuando se detecta cualquiera de los tres cambios:
 - Archivo nuevo: aparece un archivo que no existía en la línea base.

- Archivo modificado: el hash actual difiere del hash almacenado.
 - Archivo eliminado: un archivo que existía en la línea base ya no está en la copia local.
6. **Visualización y seguimiento:** los resultados se pueden consultar en la interfaz web, que presenta el número de archivos observados, alertas totales y el historial de eventos.

7.5. Sincronización remota

La sincronización remota está diseñada para asegurar que Ubuntu Desktop observe el estado real de Metasploitable2 sin depender de archivos locales de Windows. La lógica implementada es:

- Se utiliza `scp -r` para copiar recursivamente el contenido remoto.
- Se emplean opciones SSH compatibles con claves antiguas y con la versión de Metasploitable2:
 - `-oHostKeyAlgorithms+=ssh-rsa`
 - `-oPubkeyAcceptedAlgorithms+=ssh-rsa`
 - `-oStrictHostKeyChecking=no`
 - `-P 22`
- El contenido remoto se copia primero a un directorio temporal local.
- Antes de mover los datos a la carpeta de monitoreo final, el sistema borra cualquier archivo local que ya no exista en el remoto, de forma que las eliminaciones remotas se reflejen correctamente.
- Si la sincronización falla, el escaneo no se ejecuta para evitar comparaciones con datos incompletos.

8. Pruebas y validación

8.1. Requisitos del entorno

- Ubuntu Desktop para ejecutar el PoC
- Metasploitable2 como objetivo monitorizado
- Kali Linux como atacante
- Python 3 instalado en Ubuntu
- `scp` disponible en Ubuntu

8.2. Preparación de laboratorio

En Metasploitable2:

```
mkdir -p /home/msfadmin/carpeta_critica
echo "config_inicial" > /home/msfadmin/carpeta_critica/config.cfg
```

En Ubuntu:

```
cd /ruta/al/proyecto  
python3 fim_poc.py --web
```

8.3. Pasos de prueba

1. Abrir la interfaz web en <http://127.0.0.1:8000>.

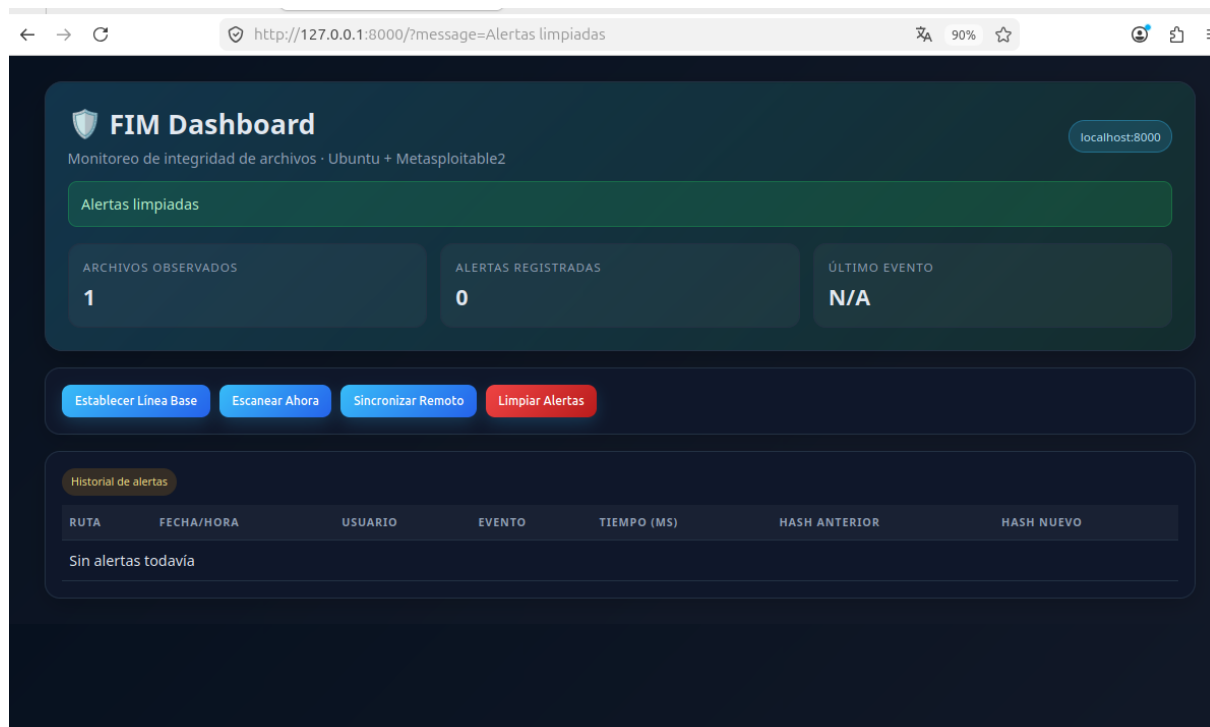


Figura 1: Vista inicial de la interfaz web en <http://127.0.0.1:8000>.

2. Pulsar **Establecer Línea Base**.

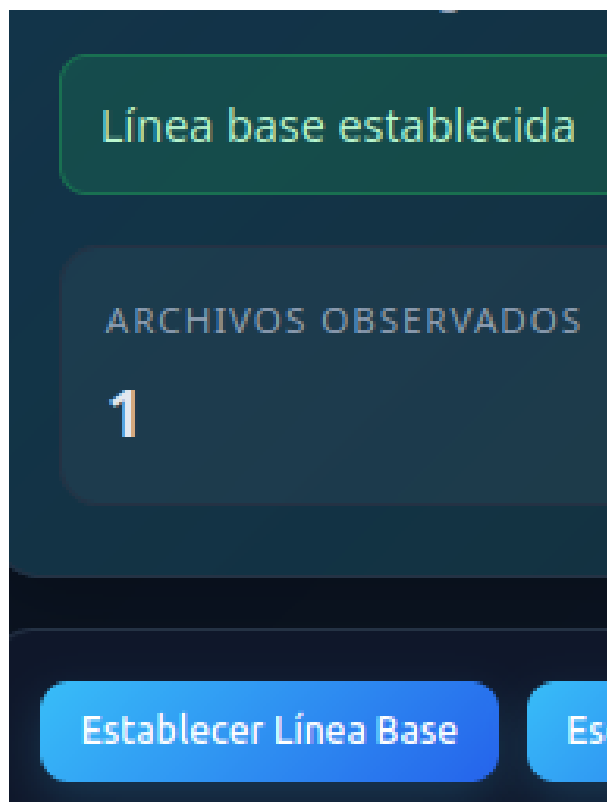


Figura 2: Confirmación de la línea base generada y archivos registrados en `inventario`.

3. Pulsar **Sincronizar Remoto**.

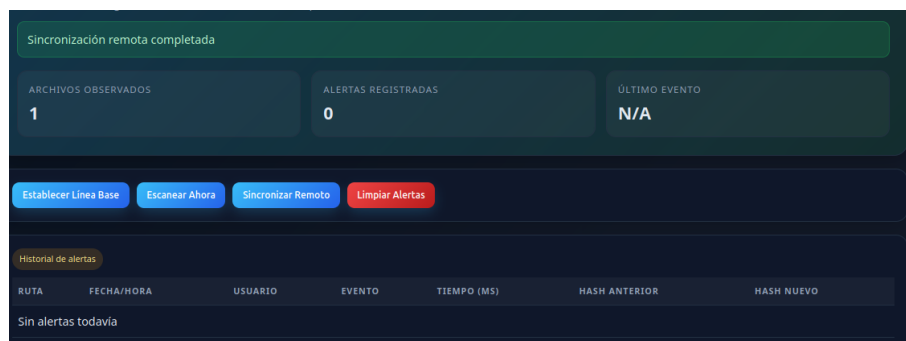


Figura 3: Salida de la sincronización remota desde Metasploitable2 hacia Ubuntu.

4. Pulsar **Escanear Ahora**.

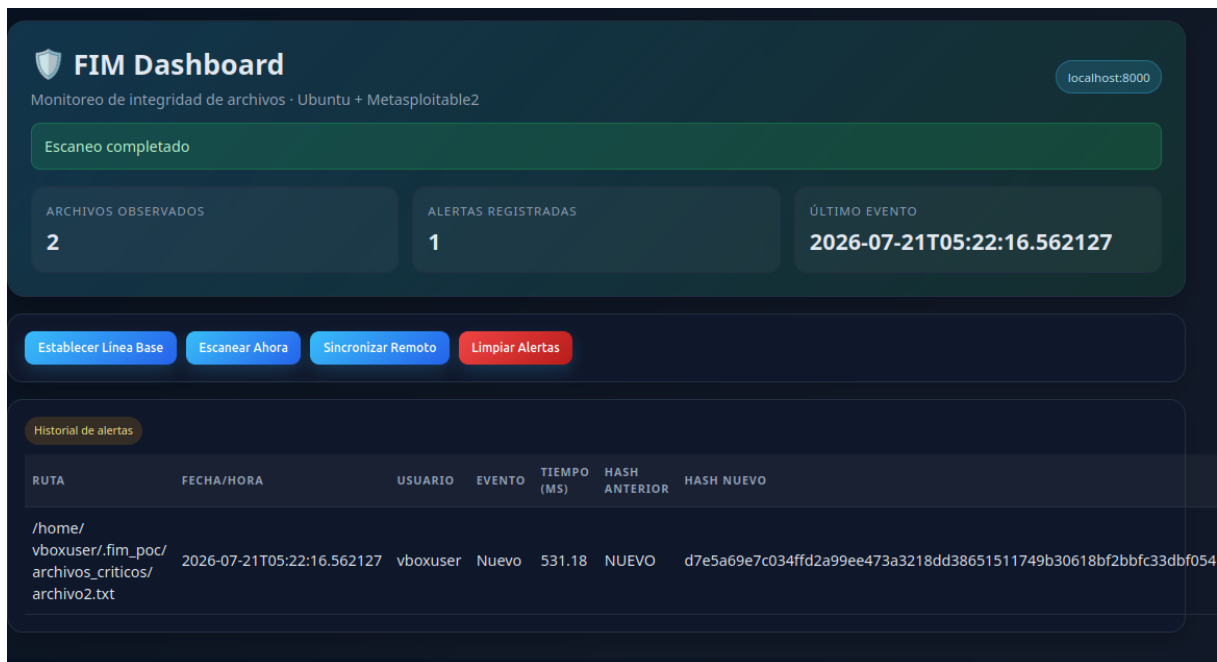


Figura 4: Estado del dashboard tras un escaneo sin modificaciones detectadas.

5. Desde Kali, ejecutar un cambio remoto vía SSH:

```
ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa -oStrictHostKeyChecking=no -p 22 msfadmin@10.0.2.4 "echo 'modificacion_maliciosa' >> /home/msfadmin/carpeta_critica/config.cfg"
```



Figura 5: Ejecución del comando de modificación remota desde Kali Linux.

6. Volver a Ubuntu y ejecutar:

- Sincronizar Remoto
- Escanear Ahora



Figura 6: Alerta generada tras detectar la modificación del archivo `config.cfg`, mostrando hash anterior y hash nuevo.

8.4. Verificación de resultados

El dashboard debe mostrar alertas en la tabla con cambios detectados. Las modificaciones deben registrar un hash anterior y un hash nuevo. Las eliminaciones deben aparecer con `hash_nuevo = ELIMINADO`.

Adicionalmente, la prueba debe validar que:

- Un escaneo posterior sin cambios no genere alertas adicionales.
- Las creaciones de archivos en Metasploitable2 se detecten tras la sincronización remota.
- Las eliminaciones remotas sean visibles en el conteo de archivos observados y en la tabla de alertas.
- Si la sincronización remota falla, el sistema indique el error y no proceda con un escaneo inválido.

9. Cómo correr el proyecto

9.1. Interfaz web

```
python3 fim_poc.py --web
```

Abrir <http://127.0.0.1:8000> en el navegador.

9.2. Modo CLI

Línea base:

```
python3 fim_poc.py --headless --action baseline
```

Escaneo:

```
python3 fim_poc.py --headless --action scan
```

Sincronización remota:

```
python3 fim_poc.py --headless --action sync
```

Monitoreo continuo:

```
python3 fim_poc.py --headless --action monitor --monitor-interval 1.0
```

9.3. Ejecución directa en caso de múltiples versiones de Python

```
python3 fim_poc.py
```

9.4. Gestión de errores y robustez

El sistema maneja condiciones de error importantes para evitar alertas falsas o datos incompletos:

- Si `scp` falla, se registra el error y no se ejecuta el escaneo.
- Si no se puede leer un archivo durante el cálculo de hash, el sistema registra un evento de error de lectura.
- Las conexiones SQLite se crean por hilo para evitar conflictos de concurrencia.
- Las rutas se normalizan antes de compararlas para evitar falsos negativos por diferencias en separadores o mayúsculas/minúsculas.

10. Cómo probar CRUD desde Kali

Crear archivo:

```
ssh -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa -oStrictHostKeyChecking=no -p 22 msfadmin@10.0.2.4 "echo 'prueba' > /home/msfadmin/carpeta_critica/nuevo.txt"
```

Leer archivo:

```
ssh -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa -oStrictHostKeyChecking=no -p 22 msfadmin@10.0.2.4 "cat /home/msfadmin/carpeta_critica/nuevo.txt"
```

Modificar archivo:

```
ssh -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa -oStrictHostKeyChecking=no -p 22 msfadmin@10.0.2.4 "echo 'línea actualizada' >> /home/msfadmin/carpeta_critica/nuevo.txt"
```

Eliminar archivo:

```
ssh -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedAlgorithms+=ssh-rsa -oStrictHostKeyChecking=no -p 22 msfadmin@10.0.2.4 "rm -f /home/msfadmin/carpeta_critica/nuevo.txt"
```

11. Resultados esperados

- El sistema debe detectar cambios en archivos existentes.
- Debe registrar la creación de nuevos archivos.
- Debe detectar y registrar eliminaciones remotas.
- El conteo de archivos observados debe coincidir con la copia sincronizada desde Metasploitable2.
- No deben generarse alertas cuando no se realizan cambios reales.
- Los tiempos de detección deben ser consistentes y menores a 2 segundos para un conjunto pequeño de archivos.
- El proceso de sincronización debe ser capaz de manejar el fallo de `scp` sin corromper la línea base.

12. Conclusiones

El proyecto demuestra una capa básica de ciberdefensa centrada en integridad de archivos. Aunque no identifica el actor (solo el evento), permite detectar alteraciones en un entorno remoto y validar la existencia de incidentes en la infraestructura monitorizada.

El diseño logra dos objetivos importantes:

- Separar el rol de monitoreo defensivo (Ubuntu Desktop) del rol de objetivo comprometido (Metasploitable2).
- Proveer un mecanismo de detección confiable sin dependencias adicionales, lo que facilita su reproducción en un entorno académico.

La solución es especialmente útil para prácticas de ciberdefensa, ya que permite observar cómo cambios introducidos desde Kali Linux se reflejan en alertas y en la base de datos del defensor.

13. Recomendaciones futuras

- Agregar autenticación y control de acceso a la interfaz web.
- Registrar origen de cambios con más metadatos.
- Expandir la detección con firmas y análisis de comportamiento.
- Incluir una vista de historial de cambios y exportación de incidentes.

14. Anexos: evidencia adicional

Esta sección reúne evidencia complementaria del laboratorio. Reemplazar cada recuadro por la captura de pantalla correspondiente (`\includegraphics`).

14.1. Topología del laboratorio

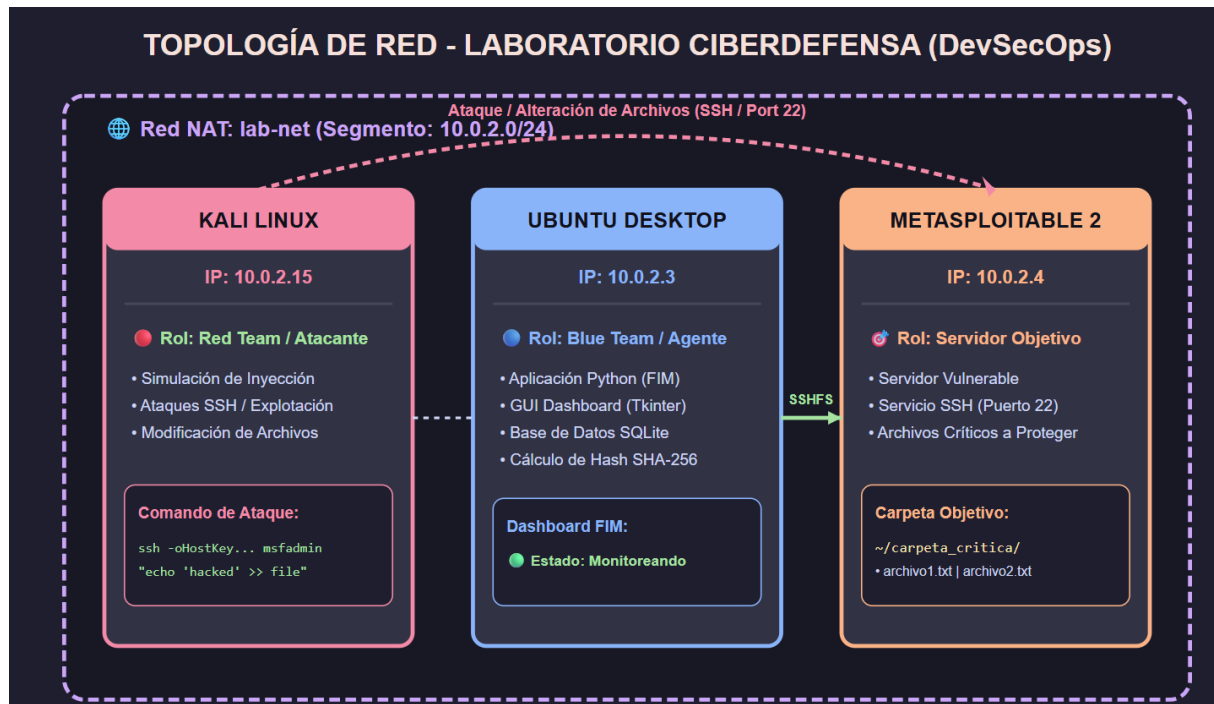


Figura 7: Topología de red utilizada en el laboratorio (direccionamiento IP de cada host).

14.2. Contenido de la base de datos

```
sqlite>
SELECT * FROM inventario;
/home/vboxuser/.fim_poc/archivos_criticos/hola.txt|6c021454bf5d1cf347f0e8a0e25c666ace886f4175eb120e1eeed0dd5c2a65e1|2026-07-21T05:22:16.597109
/home/vboxuser/.fim_poc/archivos_criticos/archivo2.txt|d7e5a69e7c034ffd2a99ee473a3218dd38651511749b30618bf2bbfc33dbf054|2026-07-21T05:22:16.581000
sqlite>
```

Figura 8: Contenido de la tabla `inventario` tras establecer la línea base.

```
SELECT * FROM alertas;
33|/home/vboxuser/.fim_poc/archivos_criticos/archivo2.txt|2026-07-21T05:22:16.562127|vboxuser|NUEVO|d7e5a69e7c034ffd2a99ee473a3218dd38651511749b30618bf2bbfc33dbf054|531.179775000055
sqlite>
```

Figura 9: Contenido de la tabla `alertas` con los eventos registrados durante las pruebas.

14.3. Ejecución en modo CLI / headless

15. Bibliografía y referencias

Referencias

- [1] Python Software Foundation. *Python 3 Documentation*. Disponible en: <https://docs.python.org/3/>.

- [2] Python Software Foundation. *hashlib — Secure hashes and message digests*. Disponible en: <https://docs.python.org/3/library/hashlib.html>.
- [3] SQLite Consortium. *SQLite Documentation*. Disponible en: <https://www.sqlite.org/docs.html>.
- [4] OpenBSD Project. *OpenSSH Manual Pages: ssh(1), scp(1)*. Disponible en: <https://www.openssh.com/manual.html>.
- [5] Ylonen, T., Lonvick, C. (eds.) *RFC 4251: The Secure Shell (SSH) Protocol Architecture*. IETF, 2006. Disponible en: <https://www.rfc-editor.org/rfc/rfc4251>.
- [6] National Institute of Standards and Technology (NIST). *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*, 2020.
- [7] National Institute of Standards and Technology (NIST). *SP 800-92: Guide to Computer Security Log Management*, 2006.
- [8] OWASP Foundation. *File Integrity Monitoring*. OWASP Cheat Sheet Series. Disponible en: <https://cheatsheetseries.owasp.org/>.
- [9] Kim, G., Spafford, E. H. *The Design and Implementation of Tripwire: A File System Integrity Checker*. Purdue University, 1993.
- [10] Rapid7. *Metasploitable2 Documentation*. Disponible en: <https://docs.rapid7.com/metasploit/metasploitable-2/>.
- [11] Offensive Security. *Kali Linux Documentation*. Disponible en: <https://www.kali.org/docs/>.
- [12] National Institute of Standards and Technology (NIST). *Cybersecurity Framework (CSF) 2.0*, 2024.

Archivo principal:	fim_poc.py
Directorio de datos local:	~/fim_poc
Base de datos:	integridad_monitores.db
Carpeta vigilada:	~/fim_poc/archivos_criticos
Host monitoreado:	10.0.2.4 (Metasploitable2)
Host atacante:	10.0.2.15 (Kali Linux)